

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Real-Time ASL Fingerspelling Recognition on Consumer Mobile Devices

By
Faruk Hankanoglu

19 October 2026

Masters Dissertation

Submitted in partial fulfilment for the award of **Master of Science in Computing** to the Department of Computer Science & Applied Physics, Atlantic Technological University (ATU), Galway.

M.Sc. in Computing

Declaration

I hereby certify that the material submitted in this dissertation towards the award of Master of Science in Computing is entirely my own work and has not been submitted for any academic assessment other than in partial fulfilment of that award.

Where material has been drawn from other sources, it has been appropriately acknowledged and cited in the text and in the list of references.

Faruk Hankanoglu 19 October 2026
G00486656

Contents

Declaration **i**

Abstract **vii**

Acknowledgements **viii**

1 Introduction **1**

1.1 Background 1
1.2 Problem statement 2
1.3 Aim and objectives 2
1.4 Research question 3
1.5 Design principles 3
1.6 Structure of this dissertation 4

2 Literature and Technology Review **5**

2.1 Introduction 5
2.2 Fingerspelling as a recognition target 5
2.3 Sensing approaches 6
2.3.1 Instrumented gloves 6
2.3.2 Depth cameras 7
2.3.3 Monocular RGB 7
2.4 Recognition methods 8
2.4.1 Appearance-based classification 8
2.4.2 Pose- and landmark-based classification 8
2.4.3 Sequence models and fingerspelling in the wild 9

2.5 Error correction over recognition output . . . 9
2.6 Dataset bias and cross-dataset evaluation . . . 11
2.7 On-device deployment . . . 12
2.8 Summary and positioning . . . 12

3 Methodology **13**
3.1 Research approach . . . 13
3.2 Datasets . . . 13
3.3 Landmark extraction and normalisation . . . 15
3.4 Classifier selection . . . 16
3.5 Generalisation experiments . . . 16
3.6 The correction layer . . . 17
3.6.1 Formulation . . . 17
3.6.2 Decoding . . . 17
3.7 Verification design . . . 18
3.8 Evaluation protocol . . . 18

4 System Design **19**
4.1 Overview . . . 19
4.2 Client-side inference . . . 20
4.3 Model export . . . 20
4.4 Temporal stabilisation . . . 20
4.5 Correction and the interface . . . 21
4.6 Deployment . . . 24

5 Evaluation **25**
5.1 Letter-level accuracy and generalisation . . . 25
5.1.1 Representation, at matched training data . . . 25
5.1.2 What the techniques achieved . . . 27
5.1.3 The structure of the residual errors . . . 28
5.2 Word-level accuracy . . . 29
5.2.1 On the reported intervals . . . 29
5.2.2 Comparison of the two classifiers . . . 30
5.2.3 Does a better classifier produce a better system? . . . 31

5.2.4 Why the augmented model was not deployed 32
5.2.5 Two properties of the metric 33
5.2.6 An internal consistency check 33
5.3 On-device performance 33
5.3.1 Where the time goes 33
5.3.2 Throughput and the throttle 36
5.3.3 Latency stability 36
5.3.4 Interpretation 37
5.4 Discussion 37
5.4.1 On comparison with published work 37
5.4.2 On whether the error pattern generalises 38

6 Conclusion **39**
6.1 Summary 39
6.2 Limitations 39
6.3 Further work 41

References **42**

A Source Code **45**

B Deployed Application **46**

C Raw On-Device Measurements **47**

List of Figures

4.1 System architecture 19
4.2 Recognition interface 21
4.3 Prediction and word builder 22
4.4 Lexical correction 23

5.1 On-device measurement panel 35

List of Tables

3.1 Datasets used 14

5.1 Cross-dataset generalisation techniques 26
5.2 Dominant letter confusions 29
5.3 Word-level accuracy of the correction layer 30
5.4 Effect of mirror augmentation 31
5.5 On-device performance 34

Abstract

Fingerspelling recognition has been demonstrated with instrumented gloves and depth cameras. Both achieve high accuracy; neither is present on the phone a person already carries. This work asks what an ordinary monocular camera can achieve — no extra sensor, no worn device, no server — and finds the constraint is not the hardware.

A classifier trained on one public American Sign Language alphabet dataset scored 99.1% on held-out images from it and 52.3% on images of the same alphabet collected by others. Training on a second dataset recovered part of that loss, reaching $71.1\% \pm 2.5$; of the techniques then evaluated — mirror augmentation, joint-angle features and convolutional transfer learning — the best reached $77.4\% \pm 5.2$, more than twenty-one points below within-dataset performance, and the worst 62.6%. With training data held constant, a convolutional classifier on pixels retained 24.2% across datasets against the landmark classifier's 52.3%: representation governs transfer more than model choice. The errors never changed shape: the same letters failed, each towards one specific substitute, and in the run reported every held-out image of M was classified as T.

This work therefore treats the error pattern as a property of the task and corrects for it downstream. A lexical correction layer treats the classifier as a noisy channel, weighting each candidate substitution by the measured probability of that confusion and by the confidence in the letter it would replace. With confusion statistics and test images drawn from disjoint halves of the held-out data — a separation adopted to avoid circularity — in-vocabulary word accuracy rose from $15.9\% \pm 2.4$ to $91.7\% \pm 2.1$ — and proved insensitive to the classifier beneath it, a six-point gain in letter accuracy producing a 0.7-point change, indistinguishable from zero.

The system is deployed as an installable progressive web application, all inference on the device. End-to-end latency was 33.2 ms on an iPhone 12 Pro and 88.5 ms on a budget Android handset, of which the 27.5 KB contributed classifier accounts for 0.14 ms and 1.30 ms.

Acknowledgements

I would like to thank my supervisor, Dr. Douglas Mota Dias, whose questions repeatedly sent me back to check things I had assumed. The direction this project eventually took — away from chasing letter accuracy and towards correcting for its failures — came out of those conversations.

I would also like to thank the academic staff of the Department of Computer Science and Applied Physics at Atlantic Technological University, Galway, whose teaching across the MSc in Computing provided the foundation for this work.

This research relied entirely on publicly available data, and I acknowledge the authors of the three datasets on which it depends: the ASL Alphabet dataset published on Kaggle; ASL-HG, published by Pranto et al. on Mendeley Data; and SignAlphaSet, published by Garg et al. on Mendeley Data. The cross-dataset experiments at the centre of this dissertation were only possible because these datasets were released openly.

I am grateful to Atlantic Technological University for access to its computing facilities and to the electronic resources through which the literature reviewed in Chapter 2 was obtained.

Finally, I would like to thank my family and friends for their encouragement over the course of the programme.

Chapter 1

Introduction

1.1 Background

American Sign Language is a complete natural language with its own grammar and lexicon, used as a primary language by a substantial population across the United States and Canada. Within it, fingerspelling occupies a particular role: a manual alphabet in which each letter is represented by a distinct handshape, used for proper nouns, technical terms, emphasis, and any word for which no established sign exists. It is a comparatively small part of signed discourse by volume, but a disproportionately important one, precisely because it carries the content that the rest of the lexicon cannot. Padden and Gunsauls [1] document both its prevalence in everyday ASL use and its grammatical integration into the language, noting that it is not a peripheral borrowing device but an established part of the lexicon.

For a hearing person with no knowledge of ASL, fingerspelling is also the most tractable entry point. The vocabulary is twenty-six items rather than thousands, the signs are largely static, and the output is ordinary written text. A system that reads fingerspelling and produces text does not solve sign language translation, and this dissertation does not claim that it does. What it can do is remove a specific, recurring friction: the situation in which a Deaf signer spells a name or a word and the hearing person opposite cannot read it.

1.2 Problem statement

The difficulty addressed here is not that fingerspelling recognition is unsolved in the laboratory. It is that laboratory results do not transfer. Published and student work alike routinely report accuracies in the high nineties, and the present work reproduced such figures without difficulty: held-out images drawn from the same dataset as the training data were classified at over 99% accuracy. The same classifier, applied to images from a different public dataset of the same alphabet, fell to 52.3%. Training on a second dataset recovered part of that loss, reaching the low seventies, but no configuration tested came close to the within-dataset figure.

This is not a subtle degradation. It is the difference between a system that works and one that does not, and it is systematically obscured by the convention of reporting accuracy on a split of a single dataset. The practical question this dissertation asks is what can be done about it — given that the obvious answer, train a better classifier, was attempted four different ways against the baseline and did not work.

1.3 Aim and objectives

The aim of this project is to build and evaluate a fingerspelling recognition system that produces usable word-level output on consumer mobile hardware under realistic cross-dataset conditions.

1. To implement a real-time fingerspelling recognition pipeline that runs entirely on a consumer mobile device without server-side inference.

2. To quantify the cross-dataset generalisation gap for landmark-based ASL alphabet classification, using datasets that are genuinely independent rather than splits of a single corpus.

3. To evaluate whether established techniques — data augmentation, rotation-invariant feature engineering, and convolutional transfer learning — close that gap.

4. To design and evaluate a confidence-aware lexical correction layer that converts an imperfect letter stream into correct words, and to verify it under an experimental design that excludes circularity.

5. To measure the deployed system's latency, throughput and model footprint on named consumer devices, establishing whether the real-time claim is supported.

1.4 Research question

Can a confidence-aware lexical correction layer recover word-level accuracy from an ASL fingerspelling classifier whose letter-level accuracy has degraded under cross-dataset conditions, within the computational budget of a consumer mobile device?

Word-level accuracy is treated as usable if it exceeds 85%, on the reasoning that below this level a user would spend more effort correcting output than the system saves. It is a design decision rather than a finding, and it is stated here, before any result is presented, so that the evaluation in Chapter 5 can be read against a criterion the reader has already seen. No claim is made that it was fixed in advance of the experiments to which it is applied.

1.5 Design principles

Two commitments shaped the work throughout and are recorded here as design principles rather than as measurable objectives, since neither admits a clean numerical criterion.

The first is *hardware accessibility*. The system was required to run on a phone the user already owns, using only its camera. No depth sensor, no glove, no external hardware. This constraint is the reason a landmark-based pipeline was chosen over approaches requiring specialised sensing, and the reason a budget Android handset was included in the device evaluation alongside a flagship.

The second is *operational independence*. All inference happens in the browser on the user's device. There is no server, no account, no network dependency after first load, and no image ever leaves the phone. Beyond the obvious privacy benefit

of not transmitting video of a user's hands, this removes the running cost that would otherwise make the system dependent on continued funding.

1.6 Structure of this dissertation

Chapter 2 reviews prior work on fingerspelling recognition, the sensing approaches that have been applied to it, the use of language models to correct recognition output, and the literature on dataset bias that frames this project's central finding. Chapter 3 describes the datasets, the recognition pipeline, the generalisation experiments and the design of the correction layer, including the measures taken to avoid circular evaluation. Chapter 4 describes the deployed application. Chapter 5 reports results at letter, word and device level. Chapter 6 assesses what was and was not achieved.

Chapter 2

Literature and Technology Review

2.1 Introduction

This review has a specific job to do. The project it supports did not arrive at its contribution by proposing a new architecture; it arrived there by failing to improve one, and then asking what the failure implied. The literature is therefore surveyed with two questions in mind. First, what have previous systems actually measured, and under what conditions — because the gap between reported and transferable accuracy is the problem this work addresses. Second, where has error correction been applied to recognition output before, and what does the present work add to that line.

The review proceeds from the linguistic subject matter to sensing approaches, then to the recognition methods built on them, then to correction, and finally to the deployment constraints that shaped the design.

2.2 Fingerspelling as a recognition target

Fingerspelling differs from lexical signing in ways that matter computationally. The alphabet is closed and small. Most letters are static handshapes, distinguished by finger configuration rather than by motion or by position relative to the body. This makes the problem approachable with single-frame classification, which is why so much work — including this project — treats it as an image or pose

classification task rather than a sequence modelling one.

Two letters resist that treatment. J and Z are dynamic: J traces a hook and Z a zigzag in the air, and neither is defined by a single frame. Datasets that present them as static images are representing a trajectory by one of its frames, usually the initial handshape. This is a known compromise and it has consequences that appear later in this work: Z is consistently among the hardest letters to classify, and the reason is not that the classifier is weak but that the label does not correspond to a well-defined static configuration.

A second difficulty is that the alphabet contains genuinely similar handshapes. The pairs M/N, and the group A/S/T/E, differ by thumb position alone, often by a few millimetres and frequently occluded from the camera's viewpoint. Shi et al. [2], working on a large corpus of fingerspelling collected from online video rather than in a laboratory, report their most frequent letter substitutions. Those pairs are not the ones measured in the present work, and the two sets in fact do not overlap; the comparison is set out in Section 5.4.2. The divergence matters for method rather than for theory: it indicates that a confusion set belongs to a particular classifier and its training data rather than to the alphabet, which is why the correction layer described here is calibrated on measured confusions rather than assumed ones.

2.3 Sensing approaches

2.3.1 Instrumented gloves

The earliest reliable sign language recognition systems instrumented the hand directly, using flex sensors, accelerometers or magnetic trackers to report joint angles without any vision problem to solve. Pugeault and Bowden [3] summarise the trade-off that followed: sensor-based capture sidesteps the occlusion and lighting problems that defeat cameras, at the cost of requiring equipment the signer must carry and wear. Accuracy is high because the ambiguity that defeats cameras — occlusion, viewpoint, lighting — does not arise.

The approach nonetheless sits awkwardly with the deployment scenario this project targets. A glove must be worn, charged, carried, and put on before a

conversation can begin, and it places the burden of preparation on the signer rather than on the hearing party who cannot read the signing. It is worth being explicit that this is an argument about deployment rather than a claim about adoption rates, for which the present author found no reliable data. The point stands on its own terms: for a system intended to be used spontaneously, by whoever happens to be present and without prior arrangement, the relevant comparison is not glove accuracy against camera accuracy, but glove accuracy against the accuracy of a system that is actually to hand at the moment it is needed. A device left at home has an effective accuracy of zero.

2.3.2 Depth cameras

Depth sensing occupies a middle position. Pugeault and Bowden [3] demonstrated real-time ASL fingerspelling recognition using a Kinect, combining appearance and depth features and publishing a dataset that remains in use. Depth resolves much of the ambiguity that monocular vision suffers, particularly for handshapes distinguished by finger extension towards or away from the camera.

The limitation is again deployment rather than accuracy. Consumer depth sensors have not become universal; where they exist on phones they are typically front-facing, restricted in API access, and absent from the budget devices that matter most for an accessibility application. A method that requires depth excludes precisely the hardware this project targets.

2.3.3 Monocular RGB

Ordinary cameras are universal, which is their entire argument. Everything else about them is worse: no depth, variable lighting, arbitrary viewpoint, and no control over background. The history of monocular fingerspelling recognition is largely a history of finding representations that survive those conditions.

2.4 Recognition methods

2.4.1 Appearance-based classification

The direct approach classifies the image. Early systems used hand-engineered descriptors; more recent ones use convolutional networks trained end-to-end on cropped hand regions. On within-dataset splits the results are excellent. Pugeault and Bowden [3] demonstrated real-time appearance-based recognition of the ASL alphabet more than a decade ago, and this is the source of the near-perfect accuracies that populate the literature and the student-project ecosystem around the public ASL alphabet datasets.

The weakness is that a convolutional classifier trained on such data learns whatever separates the classes in that data, which includes the signer's skin tone, the background, the lighting, and the camera. Torralba and Efros [4] demonstrated the general form of this problem memorably: a classifier can be trained to identify which dataset an image came from, with high accuracy, purely from incidental regularities. Their argument — that datasets carry a signature, and that models trained on one learn the signature along with the task — is the theoretical frame for the central empirical finding of this dissertation.

2.4.2 Pose- and landmark-based classification

An alternative is to separate the vision problem from the classification problem. A hand pose estimator locates keypoints; a classifier operates on their coordinates. The classifier never sees a pixel, so skin tone, background and lighting cannot influence it except insofar as they perturb the landmarks.

MediaPipe Hands [5] made this practical on mobile hardware. It uses a two-stage design — a palm detector that localises the hand, followed by a landmark model that regresses 21 keypoints within the detected region — and is trained on a combination of real annotated images and a much larger body of synthetic renderings, which is how it achieves robustness across appearance while remaining small enough to run at interactive rates on a phone.

This approach is now the dominant one for the ASL alphabet specifically. Alsharif et al. [6] combine MediaPipe keypoints with a detection network and

report a mean average precision of 98.2% at 1.3 ms per image, figures representative of the current state of the literature. That result is also representative in a second respect, and one central to this dissertation: it is obtained by training and testing on a single dataset. The present work asks what happens to such a figure when the test images come from elsewhere, a question that paper does not address.

For this project the attraction of the landmark representation is twofold. It discards appearance, which ought in principle to improve cross-dataset transfer; and a classifier consuming 63 coordinates can be very small indeed, which is what makes the deployment described in Chapter 4 possible. The first expectation proved only partly correct, and the finding is reported in Chapter 5.

2.4.3 Sequence models and fingerspelling in the wild

Work on continuous fingerspelling takes a different approach again, modelling the letter sequence rather than classifying frames independently. Shi et al. [2] introduced ChicagoFSWild, a corpus collected from naturally occurring online video rather than staged capture, and demonstrated recognition with recurrent sequence models trained with connectionist temporal classification. Their follow-up work [7] added iterative visual attention to localise the signing hand without explicit detection.

These systems address a harder problem than the present one: unsegmented, continuous, naturally produced fingerspelling, evaluated by letter error rate against a reference transcription. Their reported accuracies are far below the 99% figures of within-dataset alphabet classification, and this is not a deficiency of the methods but a reflection of what they are measuring. It is worth stating plainly that the numbers in this dissertation are not comparable to theirs, and Section 5.4 returns to this.

2.5 Error correction over recognition output

The idea that a recogniser's output should be corrected against a lexicon rather than trusted directly is old and well founded. The noisy channel model formalises it: an observed string is treated as a corrupted transmission of an intended one,

and correction becomes a search for the intended string that best explains the observation under a model of how corruption occurs. Kernighan, Church and Gale [8] applied this to spelling correction using empirically estimated confusion probabilities between characters; Brill and Moore [9] improved the error model substantially by allowing string-to-string edits rather than single-character substitutions.

Applied to fingerspelling, the channel is the classifier and the confusion statistics describe which handshapes it mistakes for which. Shi et al. [2] incorporate a character language model into their decoder. Their Table 3 reports development-set letter accuracy rising from 41.4% to 42.8% for a connectionist temporal classification model and from 35.9% to 36.7% for an encoder–decoder model. In both cases beam search and an insertion penalty are already applied to the baseline, so the increment is attributable to the language model itself: between 0.8 and 1.4 percentage points depending on the decoder. The authors characterise it as an improvement "by a small margin". It would be a misreading to conclude from this that lexical correction offers little; their setting differs from the present one in three ways that each work against large gains. Their vocabulary is open, so no dictionary constrains the output. Their metric is letter error rate over continuous sequences rather than exact-word accuracy. And their baseline recogniser operates on video, with temporal context already available to it.

The present work occupies the opposite corner of that space: a closed dictionary, an exact-word metric, and a per-letter classifier with no temporal model. These conditions are where lexical constraint has the most to offer, and the gain measured here is correspondingly large. The comparison is made explicitly in Section 5.4, because reporting a 75-point gain beside a published one-point gain without explaining the difference would be misleading.

Two features distinguish the correction layer described in Chapter 3 from a conventional spelling corrector. First, the substitution costs are not fixed edit distances but are derived from the classifier's measured confusion matrix, so that confusions the classifier actually makes are cheap to undo and ones it does not make are expensive. Second, each cost is scaled by the classifier's confidence in the letter being replaced: a letter the classifier was unsure of is cheap to overwrite, while one it was confident about resists correction. Standard spelling correctors have no access to such a signal, because a typist emits no confidence score. A

classifier does, and the evaluation in Chapter 5 indicates that this signal is worth more than the raw accuracy difference between two candidate classifiers.

2.6 Dataset bias and cross-dataset evaluation

The framing of this dissertation's central result comes from the dataset bias literature. Torralba and Efros [4] showed that widely used vision datasets are separable from one another by a classifier trained for that purpose, and that models trained on one dataset transfer poorly to another even when both purport to represent the same visual category. Their diagnostic — if a model can tell which dataset an image came from, the datasets differ in ways the task model will also learn — has proven durable.

That the effect has not been solved by scale is shown by Liu and He [10], who repeated the dataset-identification experiment on modern large-scale datasets and modern architectures. A network trained to identify which of YFCC, Conceptual Captions or DataComp an image came from reached 84.7% accuracy on held-out data, against a chance baseline of 33.3%. Thirteen years after the original diagnosis, and with datasets orders of magnitude larger, the signature remains clearly detectable. This is directly relevant to the present work: if dataset identity survives at that scale, there is little reason to expect three modestly sized ASL alphabet datasets to be free of it.

What is notable is how rarely this framing is applied to sign language recognition. The dominant evaluation practice in the ASL alphabet literature is a random split of a single dataset, which measures a model's ability to generalise across images of the same signers, in the same room, under the same lighting. That is a meaningful measurement, but it is not a measurement of whether the system will work for a different person somewhere else. The experiments in Chapter 3 were designed specifically to measure the latter.

2.7 On-device deployment

The final body of relevant work concerns getting models onto phones. The dominant approach is architectural: MobileNetV3 [11] and its relatives are designed for a favourable accuracy-to-latency ratio on mobile CPUs, and toolchains such as LiteRT, formerly TensorFlow Lite [12], quantise and compile them for deployment.

The present work takes a different route. Because the classifier consumes landmarks rather than pixels it need not be convolutional at all, and at 7,034 parameters the usual deployment machinery becomes unnecessary: the weights export as JSON and the forward pass is written by hand in JavaScript, removing the inference-runtime dependency entirely.

This inverts the usual framing of on-device machine learning. The question is not how to compress the classifier enough to fit; it is that, given a good upstream representation, the classifier was never the expensive part — a claim Section 5.3 substantiates with measurements.

2.8 Summary and positioning

Four observations from this review define the space this project occupies.

Fingerspelling recognition is well studied, and reported accuracies on single-dataset splits are high enough that further improvement there would be of little practical consequence. The dataset bias literature predicts that those accuracies overstate transferable performance, but this prediction is seldom tested in the sign language domain. Correction of recognition output against a lexicon is a mature idea with established formal foundations, but existing applications to fingerspelling operate in open-vocabulary continuous settings where its leverage is limited. And on-device deployment is treated primarily as a model compression problem, which is the wrong framing when the representation is already compact.

This dissertation tests that prediction in the fingerspelling domain, finds it confirmed, and moves the burden from the classifier to a correction layer exploiting a signal conventional spelling correction lacks: classifier confidence. The result is a system whose letter accuracy remains mediocre by published standards but whose word-level output is usable, on hardware the intended users already own.

Chapter 3

Methodology

3.1 Research approach

The work is experimental and proceeds in three stages. The first establishes the recognition pipeline and measures its performance both within and across datasets. The second attempts to close the cross-dataset gap by conventional means and records the outcome. The third, prompted by the failure of the second, designs and evaluates the correction layer.

The order matters to the argument. The correction layer is not presented as an a priori design choice but as a response to a measured failure, and the experiments that established that failure are reported in full, including the approaches that did not work.

3.2 Datasets

Three independent public datasets of the ASL alphabet were used. They were collected by different parties, under different conditions, with different signers, and this independence is the point: a split of any one of them would not have supported the generalisation measurement.

Subsampling. The counts in Table 3.1 are not the full size of each source dataset. Kaggle ASL Alphabet holds approximately 3,000 images for each of twenty-nine classes, around 87,000 in total; ASL-HG holds 1,000 for each of thirty-

Table 3.1: The three datasets, from source size to the rows actually used. "Sampled" applies the cap of 300 images per class described below; "Detected" counts the sampled images in which MediaPipe returned a hand, and "No hand" the remainder. Of the detected rows, those carrying a label outside A–Z were then excluded, leaving the rows used.

Dataset	Role	At source	Sampled	Detected	No hand	Non-letter	Used
Kaggle ASL Alphabet [13]	Train	87,000	8,700	7,348	1,352	473	6,875
ASL-HG [14]	Train	36,000	10,800	10,791	9	2,991	7,800
SignAlphaSet [15]	Held out	26,000	7,800	7,740	60	0	7,740
Training total							**14,675**

six classes, 36,000 in total; and SignAlphaSet version 3 holds 1,000 for each of twenty-six letters, 26,000 in total. Extraction from all three was capped at 300 images per class, taken in the order the filesystem returns them, with no random selection and therefore no seed.

The cap was adopted for two reasons. Running the full Kaggle dataset through MediaPipe takes several hours on the available hardware against roughly twenty minutes for the sampled subset, and more importantly it would have left the three datasets differing in size by an order of magnitude, confounding any cross-dataset comparison with a difference in training volume. At 300 per class the three are directly comparable. The results of Chapter 5 indicate that this cap is not the limiting factor: the constraint identified there is disagreement between datasets rather than quantity of data within them.

Detection failures. The "No hand" column counts sampled images in which MediaPipe returned no hand and which therefore produced no landmark vector. The three datasets differ sharply. ASL-HG lost nine images of 10,800, all of them in the digit 0, and none in any letter class. SignAlphaSet lost 60 of 7,800, a rate of 0.8%. Kaggle lost 1,352 of 8,700 overall; confining attention to the twenty-six letter classes, 7,800 images were sampled and 6,875 survived both detection and label filtering, so 925 letter images (11.9%) produced no detection. That is a substantial loss and is reported here rather than left to be inferred. It is a property of the images — the Kaggle set is tightly cropped, so a hand often extends beyond the frame edge — and not of the sampling, and it affects training data only: the held-

out set on which every cross-dataset figure is computed is SignAlphaSet, whose failure rate is an order of magnitude lower.

Non-letter rows. The non-letter column reflects the composition of each dataset rather than any processing failure. The Kaggle dataset carries three additional classes — `del`, `nothing` and `space` — accounting for its 473 excluded rows. ASL-HG is a combined alphabet-and-numeral dataset of thirty-six classes, the twenty-six letters and the digits 0–9; its excluded rows are the 2,991 detected digit examples and its retained rows the 7,800 letter examples (26×300). SignAlphaSet contains only the twenty-six letters, which is why its count is zero; this is a property of that dataset, not an absence of filtering.

Separately, and not visible in Table 3.1, a small number of source images in every dataset yielded no hand detection and so produced no landmark vector at all. For SignAlphaSet this was 60 of 7,800 source images, a detection failure rate of 0.8%. Such images are excluded before the counts above are formed.

SignAlphaSet was designated the held-out set before any experiments were run and was never used for training, hyperparameter selection, or model choice.

3.3 Landmark extraction and normalisation

Each image is processed by the MediaPipe hand landmarker [5, 16], which returns 21 three-dimensional landmarks. Images in which no hand is detected are discarded.

The raw coordinates are unusable directly, because they encode where the hand was in the frame and how large it appeared, neither of which is relevant to which letter is being signed. Two normalisations are applied. The wrist landmark is translated to the origin, removing absolute position. All coordinates are then divided by the distance from the wrist to the middle fingertip, removing scale and therefore the hand's distance from the camera and the signer's hand size.

What this does *not* remove is rotation. A hand tilted relative to the camera produces a different coordinate vector for the same handshape. This is a deliberate limitation of the representation and it motivated one of the generalisation experiments described in Section 3.5.

The resulting feature vector is 63 values: 21 landmarks in three dimensions.

3.4 Classifier selection

Two classifiers were evaluated: a random forest and a small feed-forward neural network. The network has 63 inputs, two hidden layers of 64 and 32 units with ReLU activation and 0.2 dropout, and a 26-way softmax output — 7,034 parameters in total. It is trained with the Adam optimiser for 60 epochs using sparse categorical cross-entropy.

Both were evaluated under identical conditions in Section 5.2. The neural network was carried forward to deployment; the reasoning, which rests on deployment cost rather than on a clear accuracy advantage, is given there.

3.5 Generalisation experiments

Five approaches were evaluated against the cross-dataset gap.

1. **Baseline.** Train on one dataset, test on another, with no mitigation.
2. **Combined training.** Train on two datasets jointly, test on the third. Exposure to two sources of variation should in principle teach the classifier to ignore source-specific regularities.
3. **Mirror augmentation.** Negating the x coordinates produces a valid left-handed example from a right-handed one, doubling the training set and removing handedness as a confound.
4. **Rotation-invariant features.** Rather than coordinates, 19 joint angles were computed, which are invariant to rotation of the whole hand. This directly addresses the limitation identified in the normalisation scheme.
5. **Convolutional transfer learning.** Hands were cropped from the source images and a MobileNetV3-Small [11] backbone was fine-tuned on them, abandoning the landmark representation entirely to test whether the bottleneck lay there.

Results are reported in Section 5.1. All five are reported, not only the best, since selecting the best-performing technique on the test set and reporting only that figure would constitute a selection bias.

3.6 The correction layer

3.6.1 Formulation

The classifier is treated as a noisy channel. Let $f_{c,p}$ denote the probability that the classifier predicts letter p when the true letter is c, estimated empirically. For an observed letter p with classifier confidence $\text{conf}(p)$, the cost of substituting candidate letter c is

$$\text{cost}(p \to c) = (1 - f_{c,p}) \cdot \text{conf}(p) \tag{3.1}$$

with $\text{cost}(p \to p) = 0$.

Equation 3.1 encodes two ideas. The first factor makes a substitution cheap when the classifier is known to confuse those two letters and expensive otherwise, so the correction follows the classifier's actual error structure rather than an abstract notion of similarity. The second factor scales that cost by how confident the classifier was: a letter the classifier hedged on is cheap to overwrite, while one it asserted strongly resists correction even if the substitution is otherwise plausible.

3.6.2 Decoding

For an observed string of length n, every dictionary word of the same length is scored as the sum of per-position costs, and the cheapest is selected. If its mean cost per letter exceeds a threshold of 0.5, no correction is returned and the raw string stands.

The dictionary is derived from a frequency-ordered list of the 10,000 most common English words. Entries containing characters outside A–Z were removed, and the list was restricted to words of three to eight letters, leaving 7,228 words.

Both bounds follow from the evaluation rather than from the application. Words of one or two letters are excluded because at those lengths the correction problem is degenerate: almost every two-letter string is itself a dictionary word, so the layer has nothing to correct and the measurement would be dominated by trivially correct cases. Words longer than eight letters are excluded because the uncorrected baseline for them is effectively zero at the measured letter accuracy

($0.71^9 = 4.6\%$, falling to 3.3% at ten letters), which would compress the comparison between raw and corrected accuracy rather than inform it. An earlier evaluation protocol used a wider range of two to nine letters over approximately 8,500 words; the figures reported in this dissertation come from the narrower range throughout, and were produced by the evaluation script at tag `v2.1-dissertation` of the repository given in Appendix A.

3.7 Verification design

The correction layer poses an obvious circularity risk. The confusion matrix $f_{c,p}$ describes how the classifier errs; if it were estimated on the same images used to evaluate word accuracy, the correction would be tuned to the specific errors it is then credited with fixing, and the result would be meaningless.

The held-out dataset is therefore partitioned into two disjoint halves, stratified by letter. Half A — 3,869 images — is used solely to estimate the confusion matrix. Half B — 3,871 images — is used solely for word evaluation. No image appears in both. The correction layer is thus evaluated on errors it has never observed, drawn from the same distribution as those it was calibrated on.

3.8 Evaluation protocol

Words are sampled uniformly at random from the dictionary. For each letter of each sampled word, one image of that letter is drawn at random from Half B, and the classifier's prediction and confidence for that image are recorded. The resulting predicted string is the system's raw output; the correction layer is then applied. A word counts as correct only on exact match.

This is repeated for ten seeds of 300 words each. Two variants are reported. In the first, the classifier is trained once and the seeds vary only the sampled words and the images drawn; the resulting intervals capture sampling variability alone. In the second, the classifier is retrained from a fresh initialisation on every seed, so the intervals also capture variability attributable to the classifier. The distinction is not cosmetic: the intervals differ by a factor of more than three, as Section 5.2 shows.

Chapter 4

System Design

4.1 Overview

The deployed system is an installable progressive web application. All computation — camera capture, landmark extraction, classification, temporal stabilisation and lexical correction — happens in the browser on the user's device. There is no server component and no network traffic after the initial load.

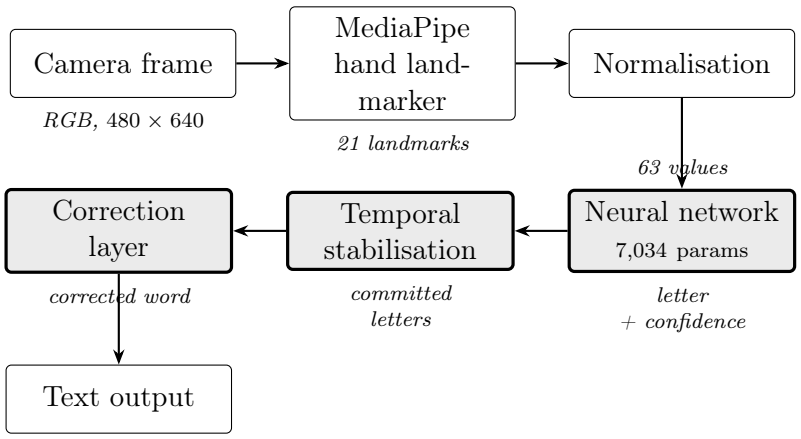


Figure 4.1: The recognition pipeline. Shaded components were implemented for this project; unshaded components are the camera and the off-the-shelf landmark extractor. Every stage executes on the user's device inside the browser; no data leaves the phone.

Figure 4.1 shows the arrangement. The division it depicts matters for inter-

preting the performance results: the landmark extractor is an external dependency that dominates per-frame cost, while the three contributed components together account for a fraction of a millisecond (Section 5.3).

4.2 Client-side inference

The MediaPipe hand landmarker runs in the browser through its WebAssembly build, with GPU delegation where available. Inference is invoked on a loop driven by animation frame callbacks and throttled to one cycle per 80 ms; the reasoning behind that interval, and its measured effect, are given in Section 5.3.

4.3 Model export

The trained network is exported as JSON: for each layer, a weight matrix, a bias vector and an activation name. The forward pass is implemented directly in JavaScript as a sequence of matrix-vector products with ReLU and softmax, requiring no inference runtime.

The port was verified numerically rather than assumed correct. The JavaScript implementation and the original Python model were evaluated on identical inputs and their outputs agreed to within 10^{-21}, which establishes that the deployed classifier is the classifier that was evaluated.

4.4 Temporal stabilisation

A per-frame classifier applied to a live camera produces an unusable stream of letters. The system therefore commits a letter only when the same prediction persists, above a confidence floor of 0.55, for a continuous window of 1.4 seconds, followed by a 600 ms refractory period before the next letter can be committed.

The 1.4 second window is the key design parameter, and it drives the throughput decision discussed in Section 5.3: at the measured rate of approximately ten inference cycles per second, each committed letter rests on roughly fourteen consecutive predictions.

4.5 Correction and the interface

Committed letters accumulate in a word buffer. When the user indicates the word is complete, the correction layer runs and the interface presents both the raw string and the corrected result, together with alternative candidates that the user can select instead. Showing the raw prediction alongside the correction is a deliberate choice: the system does not conceal what it actually recognised, which matters both for user trust and for the user's ability to detect a wrong correction.

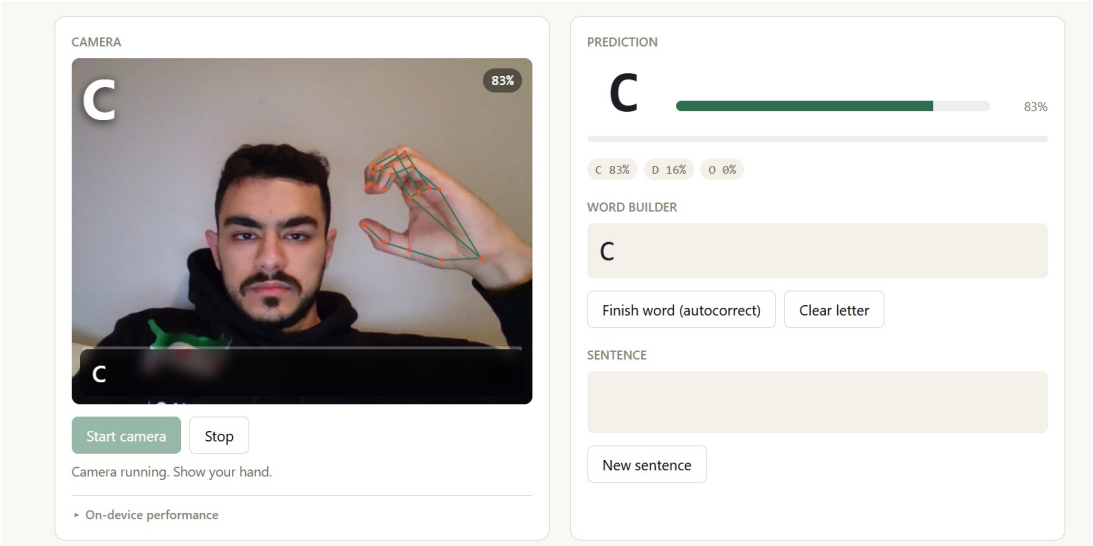


Figure 4.2: The recognition interface. The detected hand skeleton is drawn over the camera feed, and the predicted letter appears both as an overlay on the frame and in the prediction panel beside it. Here C is recognised at 83 % confidence; the runner-up is D at 16 %, which is the letter C is most often confused with in the cross-dataset evaluation (Table 5.2). The interface is shown here in a desktop browser, where the wider viewport places the camera and prediction panels side by side and the text is legible at print size; Figure 5.1 shows the same application running on the iPhone 12 Pro.

Figures 4.2 to 4.4 show the interface in use. Presenting the raw sequence beside the correction, rather than silently substituting one for the other, is a deliberate choice: the user can see when the correction layer has intervened, and can override it from the candidate list when it has chosen wrongly.

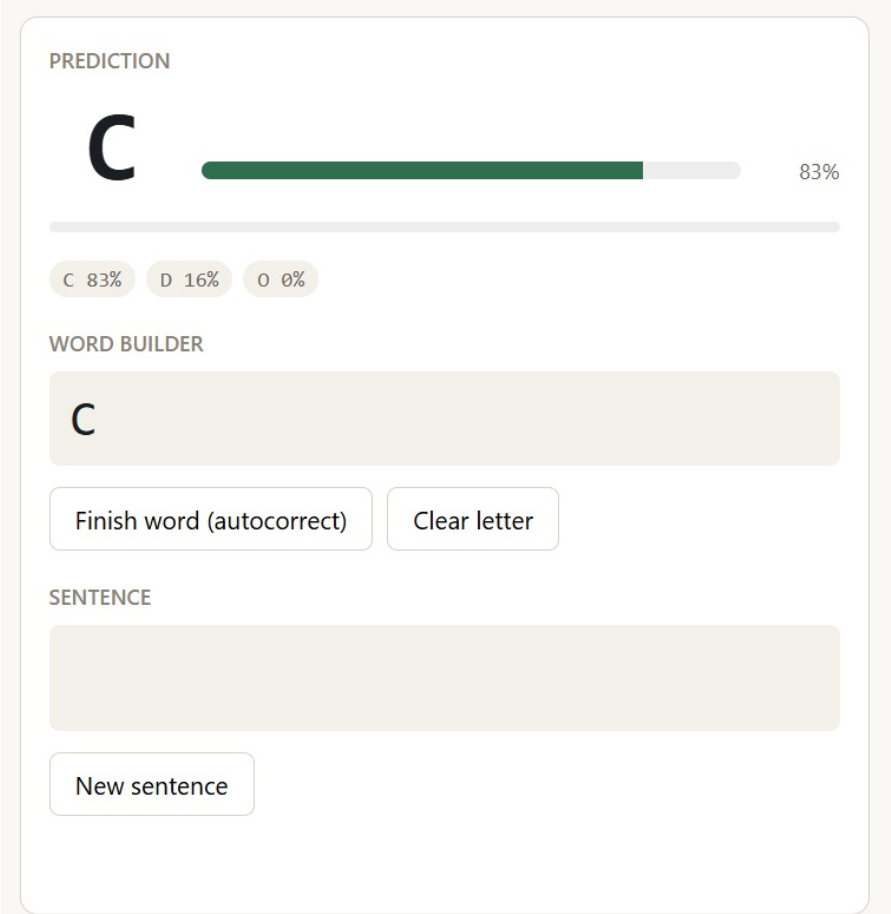


Figure 4.3: The prediction and word-builder panels. The current letter is shown with its confidence and the two next most likely alternatives — here C at 83 %, against D at 16 % and O at 0 % — and committed letters accumulate in the word buffer below. Showing the alternatives and the raw buffer is deliberate: the system does not conceal what it actually recognised.

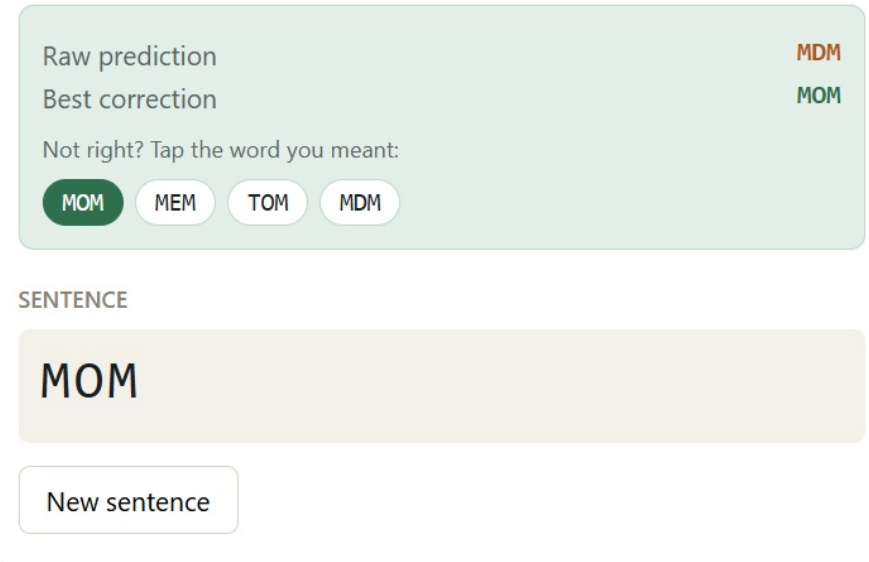


Figure 4.4: The correction step. The raw letter sequence produced by the classifier is displayed alongside the corrected word, with alternative dictionary candidates offered beneath, ranked by the channel cost of Equation 3.1. The signer spelled MOM and the classifier read the O as D — the dominant substitution for that letter in Table 5.2, occurring in 43.5% of held-out images of O — and the correction layer recovered the intended word. Of the alternatives offered, TOM reflects the M-to-T confusion from the same table, and MDM is the uncorrected sequence, retained as a candidate in case the raw reading was right.

4.6 Deployment

The application is distributed as a progressive web application: a web manifest declares it installable, and a service worker caches the application shell so that it launches offline. On Android it installs to the home screen and launches without browser chrome.

It is also packaged for store distribution and published on Google Play as *Fingerspell — ASL Reader*, listed in Appendix B. The system is therefore available both as an installable web application served from the deployment URL and through the Android store.

The store package was produced with PWABuilder, which wraps the deployed origin as an Android trusted web activity rather than reimplementing the interface natively. The package contains no copy of the recognition code: it launches the same origin, so the classifier, the correction layer and the service worker are the same files in both routes. One consequence is that redeploying the web application updates existing store installations without a new release, which is why the two routes cannot diverge.

The deployment serves no Digital Asset Links file: the published application contains no `.well-known/assetlinks.json`. Verification of the package against the domain is therefore not established. This affects only how the store build frames the page; it does not affect the camera, the recognition pipeline, or any measurement reported in this dissertation.

The figures in Section 5.3 were measured in the browser; the store build runs the same page in the same engine, so they are expected to carry over, but this was not measured separately.

Chapter 5

Evaluation

5.1 Letter-level accuracy and generalisation

Within-dataset performance replicates the published pattern: trained and tested on splits of the same dataset, the classifier exceeds 99% accuracy. Tested on a dataset collected elsewhere it reaches 52.29%. The remainder of this section quantifies that difference and reports what was done about it.

5.1.1 Representation, at matched training data

Block (a) of Table 5.1 isolates the one comparison this project's architecture depends on. All three models are trained on the same single dataset and tested on the same unseen one; only the representation differs.

Both approaches perform comparably within the training dataset — 99.05% for landmarks, 96.07% for pixels. Across datasets they diverge sharply. The landmark classifier retains 52.29%; the convolutional classifier retains 24.17%, a loss of 71.9 points against the landmark pipeline's 46.8. At matched training data the landmark representation transfers more than twice as well.

The manner of the convolutional model's failure is as informative as its magnitude. Eight of the twenty-six letters have zero recall on the held-out data — the model never predicts them at all — while Y is predicted for every one of its own instances and for enough others that its precision falls to 0.27. A model that has learned handshape degrades gracefully; a model that has learned the appearance

Table 5.1: Techniques evaluated against the cross-dataset gap, all tested on the held-out SignAlphaSet dataset (7,740 images). The upper block holds training data constant at a single dataset and varies the representation; the lower block holds the landmark representation constant and varies the training strategy. Figures are single runs except where noted.

		Within-dataset	**Cross-dataset**
(a) Single training dataset (Kaggle), varying representation			
Landmarks	neural network	99.05 %	52.29 %
Landmarks	random forest	99.12 %	42.49 %
Pixels	MobileNetV3-Small†	96.07 %	24.17 %
(b) Two training datasets (Kaggle + ASL-HG), landmark representation			
Combined training‡	neural network	—	71.10 %
Combined training‡	random forest	—	73.69 %
+ mirror augmentation‡	neural network	—	**77.42 %**
Joint-angle features	neural network	—	62.55 %

† Within-dataset figure is validation accuracy on a held-out split of the training data after fine-tuning, not an independent test split.

‡ Mean over ten retrained seeds, ±2.49 for combined training and ±5.19 for mirror augmentation (Tables 5.3 and 5.4).

statistics of one dataset collapses onto whichever classes those statistics still favour when the appearance changes. This is the concrete form of the dataset-signature effect described by Torralba and Efros [4] and Liu and He [10].

One qualification is necessary. The convolutional model was trained with a conventional two-phase transfer-learning schedule — eight epochs with the backbone frozen, then eight with it unfrozen — on 6,875 cropped images, and was not subjected to extensive hyperparameter search, architecture selection or photometric augmentation. A more carefully optimised convolutional model would very likely transfer better than 24.17%. The claim supported by this experiment is therefore the narrower one: under matched and modest training budgets, the landmark representation transferred more than twice as well as pixels. It is not a claim that convolutional approaches cannot work, and the comparison should not be read as establishing an upper bound on them.

With that qualification, the result remains the empirical justification for preferring landmarks in Chapter 4, and it is reinforced by the fact that the landmark pipeline achieved its advantage with a classifier of 7,034 parameters against a backbone of several million.

5.1.2 What the techniques achieved

Mirror augmentation performed best, at $77.42\% \pm 5.19$ over ten retrained seeds. An initial single run gave 75.18%, which against the combined-training standard deviation of 2.49 percentage points (Section 5.2) sat approximately 1.6 standard deviations above the combined-training mean of 71.10%. That was suggestive of a real improvement but does not establish one on its own. The experiment was therefore repeated across ten retrained seeds under the full evaluation protocol; the outcome is reported in Section 5.2.3, which finds the improvement in letter accuracy statistically significant but without measurable effect on word accuracy.

Rotation-invariant joint angles performed worst, at 62.55%. Reducing 63 coordinates to 19 angles removes the sensitivity to hand orientation that motivated the change, but it also discards information the classifier was using. The trade was unfavourable.

The baseline establishes the scale of the problem. A landmark classifier trained

on a single dataset scores 99.05% on held-out images from that same dataset and 52.29% on images of the same alphabet collected elsewhere — a loss of nearly forty-seven points from a change of source alone, with no change to the task, the alphabet, or the model. The equivalent convolutional model loses seventy-two.

Training on two datasets instead of one recovers roughly nineteen of those points, which is the largest single improvement obtained and the least sophisticated. The techniques attempted afterwards moved the figure between 8.6 points below the combined-training baseline, for joint-angle features, and 6.3 points above it, for mirror augmentation.

What no technique achieved was closing the gap. From the best configuration to within-dataset performance remains a distance of roughly twenty-two points (99.05% against 77.42%). A problem that responds strongly to simply adding a second data source, weakly to augmentation, negatively to a change of feature representation, and catastrophically to abandoning landmarks for pixels — while remaining more than twenty points short throughout — is being limited by the data rather than by the model.

5.1.3 The structure of the residual errors

The decisive evidence is not the aggregate accuracy but the shape of the errors, which is remarkably stable across techniques.

Under mirror augmentation, the best-performing configuration, the errors are not diffuse. Several letters are misclassified almost in their entirety, and each is misclassified as one specific other letter. Table 5.2 reports the dominant confusion for each of the affected letters.

In that run all 287 held-out images of the letter M were classified as T, a complete substitution rather than a tendency. A classifier that has learned a decision boundary for M would be expected to classify at least some of them correctly; one whose training data defines M differently from the test data would not.

This interpretation is supported by the fact that the same letters fail under a different feature representation. In the joint-angle experiment, which discards coordinates entirely in favour of 19 angles, G, M and X again have recall at or near

Table 5.2: The dominant confusions under the best-performing technique (mirror augmentation). Each row reports the single most frequent misclassification of that letter, as a proportion of all held-out images of it. These counts come from one run, the first of the ten retrained seeds reported in Section 5.2.3, and not from a mean over the ten; the seed-to-seed spread of 5.19 percentage points in letter accuracy means the proportions would differ somewhat from seed to seed.

True letter	Predicted as	Proportion
M	T	100.0 %
G	Z	93.0 %
C	D	89.5 %
N	T	57.3 %
O	D	43.5 %
X	D	36.0 %

zero. Two representations sharing no features in common fail on the same letters, which locates the problem upstream of the representation.

Direct visual comparison of images of the affected letters across the datasets confirmed it: the handshapes themselves differ. The datasets do not entirely agree on what some letters look like. Under that condition no classifier can be simultaneously correct on both, and the observed ceiling is a property of the data rather than of the model. This is the finding that motivated the correction layer: if the confusions are systematic and stable, they can be modelled rather than eliminated.

5.2 Word-level accuracy

The correction layer raises in-vocabulary word accuracy from 15.9% to 91.7%, a gain of 75.8 percentage points, and exceeds the 85% threshold stated in Section 1.4.

5.2.1 On the reported intervals

All intervals are sample standard deviations across ten seeds. Two variants were computed because they measure different things. With the classifier trained once and seeds varying only the sampled words and images, the interval on letter ac-

Table 5.3: Word-level accuracy of the confidence-aware correction layer on the held-out dataset, for both classifiers under identical evaluation code. Means over ten seeds with sample standard deviations; the classifier is retrained from a new initialisation on each seed, so the intervals include classifier variability as well as word-sampling variability. Each seed samples 300 words uniformly from a 7,228-word dictionary. Confusion statistics and evaluation images are drawn from disjoint halves of the held-out set (3,869 and 3,871 images).

	Neural network	**Random forest**
Letter accuracy (held-out)	71.10 ± 2.49 %	73.69 ± 1.65 %
Word accuracy, uncorrected	15.87 ± 2.38 %	18.44 ± 2.91 %
Word accuracy, corrected	91.67 ± 2.05 %	90.07 ± 1.14 %
Absolute gain	+75.80 pp	+71.63 pp
Candidates accepted	100 %	100 %
Deployed	yes	no

curacy is ±0.75 percentage points. With the classifier retrained on every seed, it is ±2.49 — a factor of 3.3 wider. The narrower figure understates the true uncertainty because it holds fixed a source of variation that a user of the system would experience. The retrained figures are therefore the ones reported in Table 5.3 and in the abstract.

5.2.2 Comparison of the two classifiers

The comparison is not one-sided. The random forest is the better letter classifier, by 2.59 percentage points ($p = 0.015$). It is correspondingly better on uncorrected words. Yet the neural network finishes ahead after correction, by 1.60 percentage points, with a correction gain 4.2 points larger.

Two cautions apply. First, these are three tests on the same data; under a Bonferroni correction only the letter-accuracy difference remains significant, and the word-level differences should be described as suggestive rather than established. Second, an earlier single-initialisation comparison appeared to show the network ahead by 4.87 points; that figure was an artefact of an unlucky forest initialisation and is not reported.

The explanation consistent with these results is that the correction layer de-

pends on the quality of the confidence estimate, not only on the accuracy of the prediction. Since Equation 3.1 weights every substitution by $\mathrm{conf}(p)$, a classifier whose confidence tracks its correctness supplies more useful information to the decoder than one whose confidence does not, independently of how often each is right. The softmax distribution of the network appears to serve this purpose better than the vote proportions of the forest. This interpretation fits the evidence but was not tested directly; a calibration analysis would be required to establish it, and is noted in Section 6.3.

The network was nonetheless carried forward primarily on deployment grounds. It is 7,034 parameters and 27.5 KB, and evaluates in 0.14 ms on an iPhone 12 Pro. A 200-tree forest cannot be reduced to a comparable artefact without either shipping a far larger serialised model or implementing a tree-walking interpreter in the browser, neither compatible with the single-file deployment described in Chapter 4.

5.2.3 Does a better classifier produce a better system?

The augmentation experiment of Table 5.1 was repeated over ten retrained seeds so that it could be compared against the deployed configuration under the identical protocol. The result is the most informative single comparison in this work.

Table 5.4: Mirror augmentation against the deployed configuration, both retrained over ten seeds under identical evaluation. Significance from Welch's t-test.

	Deployed	**+ mirror aug.**	**Difference**
Letter accuracy	$71.10 \pm 2.49\,\%$	$77.42 \pm 5.19\,\%$	$+6.32$ pp ($p = 0.004$)
Word, uncorrected	$15.87 \pm 2.38\,\%$	$26.17 \pm 8.44\,\%$	$+10.30$ pp ($p = 0.004$)
Word, corrected	$91.67 \pm 2.05\,\%$	$92.37 \pm 2.28\,\%$	$+0.70$ pp ($p = 0.48$)

Mirror augmentation improves the classifier substantially and significantly: letter accuracy rises by 6.32 percentage points and uncorrected word accuracy by 10.30, both at $p = 0.004$. After correction, the difference is 0.70 percentage points and is not statistically distinguishable from zero ($p = 0.48$).

The correction layer absorbs the improvement. A six-point gain in letter accuracy produces no measurable gain in the output the user actually receives.

This is a substantive finding rather than a disappointing one, and it supports the central argument of this dissertation from the opposite direction to the earlier results: where Section 5.1 showed that classifier accuracy could not be improved enough to matter, this shows that improving it further would not have mattered either. Over the range examined, end-to-end performance is governed by the correction layer and not by the classifier feeding it.

Two consequences follow. The first is a ceiling: if word accuracy is insensitive to a six-point change in letter accuracy, the binding constraint near 92% lies in the decoder — the dictionary, the cost function, and the requirement of an exact match — rather than in recognition. Further effort directed at the classifier would be misallocated. The second concerns deployment, and is discussed below.

5.2.4 Why the augmented model was not deployed

Table 5.4 might appear to argue for retraining the deployed model with augmentation. Two observations argue against it.

The end-to-end benefit is not measurable: word accuracy, which is what the application produces, differs by less than one percentage point and is well within seed-to-seed variation.

More importantly, augmentation makes training markedly less stable. The standard deviation of letter accuracy across seeds widens from 2.49 to 5.19 percentage points, and that of uncorrected word accuracy from 2.38 to 8.44 — factors of 2.1 and 3.5. The per-seed uncorrected word accuracies under augmentation, sorted, are 17.0, 17.0, 17.3, 18.3, 24.7, 27.0, 33.0, 34.0, 35.7 and 37.7 %: four seeds cluster near 17% and four near 35%, with two between. Augmented training appears to settle into one of two distinct regimes depending on initialisation.

A deployed model is a single draw from that distribution. Selecting a favourable draw by evaluating candidates on the held-out set would be the selection bias that Section 5.1 is at pains to avoid, and accepting an arbitrary draw means accepting a materially worse model roughly half the time. The more consistent unaugmented configuration was therefore retained, for a difference in expected output quality that the experiment shows to be negligible. Resolving the bimodality — rather than exploiting it — is noted in Section 6.3.

5.2.5 Two properties of the metric

The acceptance threshold rejected no candidate in this evaluation. Because correct letters contribute zero cost and substitution costs are scaled by confidence, the mean cost per letter remained below 0.5 for every sampled word. The threshold is therefore a safeguard against correcting input with no plausible dictionary analogue, not a component contributing to the accuracy reported here.

Second, and more consequentially, the metric is in-vocabulary by construction. Words are sampled from the dictionary, so every target is guaranteed to be a candidate. This isolates the correction mechanism, which is what the experiment is for, but it is an upper bound on open-vocabulary performance. A word outside the dictionary cannot be produced at all.

5.2.6 An internal consistency check

If letter errors were independent, a word of mean length L would be correct with probability 0.71^L. The mean length over the 7,228-word dictionary is 5.75 letters, which predicts 14.1%, against 15.87% observed. The two agree to within two percentage points, indicating that letter errors within a word are close to independent, which is the assumption the channel model relies on.

5.3 On-device performance

Two devices were measured, chosen to bracket the hardware range the system targets: a flagship phone and a budget Android handset retailing below €200.

Measurements were taken from the instrumentation shown in Figure 5.1, which is built into the application itself.

5.3.1 Where the time goes

The dominant cost on both devices is landmark extraction: 33.1 ms and 87.2 ms. Classification required 0.14 ms and 1.30 ms — factors of approximately 236 and 67 below the feature extraction preceding it, and between 0.42% and 1.47% of frame time.

Table 5.5: On-device performance of the deployed application. Latency statistics are computed over a rolling window of the final 120 inference cycles, after discarding the first 20 as warm-up, during approximately 35 s of continuous operation with a hand in frame. Landmark extraction is reported as end-to-end latency minus classification latency so that all three figures describe the same frames.

	iPhone 12 Pro	**Samsung Galaxy A22**
Platform		
Operating system	iOS 26.2.1	Android 14
Browser	Safari 26.2	Chrome 150
Camera resolution	480×640	480×640
Latency per frame		
Landmark extraction	33.1 ms	87.2 ms
Classification	0.14 ms	1.30 ms
End-to-end, median	33.0 ms	82.2 ms
End-to-end, mean	33.2 ms	88.5 ms
End-to-end, 95th percentile	37.0 ms	110.5 ms
End-to-end, worst observed	38.0 ms	248.8 ms
Throughput		
Nominal inference rate	12.5 Hz (80 ms interval)	
Measured throughput	10.0 fps	10.7 fps
Unthrottled capacity	30.1 fps	11.3 fps
Duty cycle	33.2 %	94.7 %
Limiting factor	throttle	compute
Model footprint		
Classifier parameters	7,034	
Classifier weights (float32)	28,136 bytes (27.5 KB)	
Landmark model (MediaPipe, float16)	≈7.5 MB	

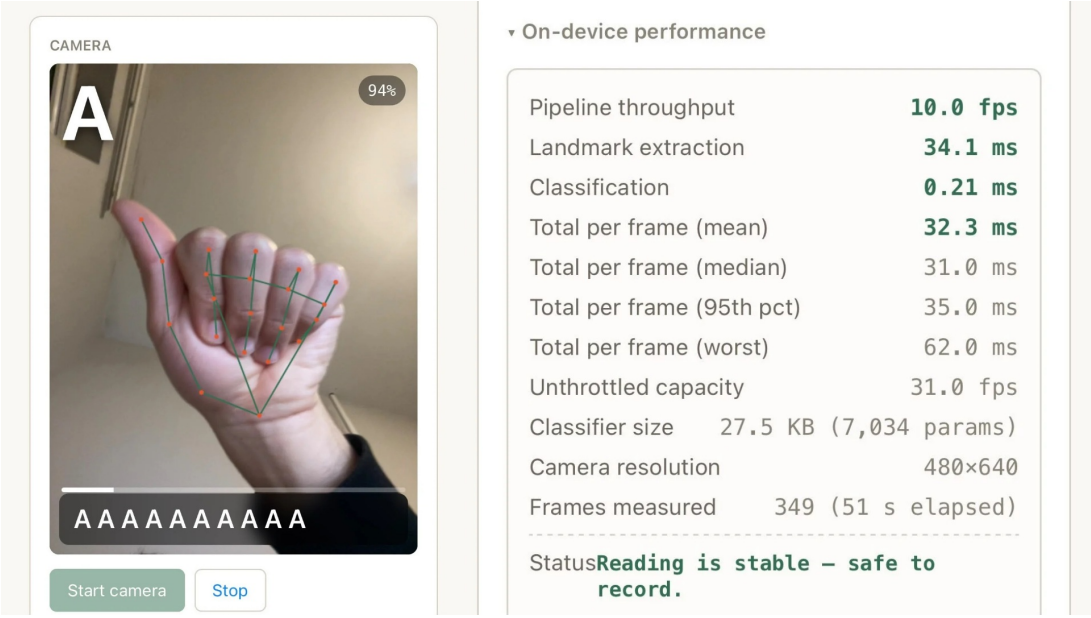


Figure 5.1: The measurement panel built into the application, captured on the iPhone 12 Pro held in landscape. Statistics are computed over a rolling window of the last 120 inference cycles after discarding the first 20 as warm-up, and the panel reports whether the reading has stabilised before it should be recorded — the status line beneath the frame count. The figures in Table 5.5 were read from this panel on the two devices listed there, in the sessions reproduced verbatim in Appendix C; the reading visible here is from a separate session and is not expected to match them exactly.

This is the principal quantitative justification for the architecture of Chapter 4. Because the classifier consumes 63 normalised coordinates rather than pixels, its cost is negligible against a landmark extractor the pipeline requires in any case. An image-based convolutional classifier would have replaced the 33–87 ms stage with a comparable or greater cost rather than adding a fraction of a millisecond to it.

5.3.2 Throughput and the throttle

Inference is throttled to one cycle per 80 ms. Because the throttle is evaluated within animation frame callbacks, which fire at the display refresh interval of approximately 16.7 ms, the effective rate is lower than the nominal 12.5 Hz: 10.0 and 10.7 fps were measured. At these rates the 1.4 second stability window accumulates fourteen to fifteen consecutive predictions per committed letter, which is ample for the stability criterion the word builder applies, and a higher rate would add no discriminative evidence while increasing power draw.

The two devices relate to the throttle in opposite ways, and the comparison is informative. The iPhone completes a cycle in 33.2 ms within a 100 ms period — a duty cycle of 33.2% — and its unthrottled capacity of 30.1 fps is three times its configured rate; it is throttle-bound and substantially idle. The A22 requires 88.5 ms, exceeding the 80 ms interval, so it never waits at the gate and runs continuously at a 93.5 ms period, a duty cycle of 94.7%. It is compute-bound, with only 6% headroom. This accounts for the apparent anomaly that the slower device reports the marginally higher frame rate.

5.3.3 Latency stability

Stability differs sharply between the devices. On the iPhone the distribution is tight: a median of 33.0 ms, a 95th percentile of 37.0 ms, and a worst observed frame of 38.0 ms, only 15% above the median. The A22 exhibits a pronounced right tail — a 95th percentile of 110.5 ms and a worst frame of 248.8 ms, three times its median — and its mean is correspondingly inflated above its median. The median is the more representative statistic for that device.

These occasional stalls degrade the smoothness of the landmark overlay but do not cause recognition errors, because letter commitment integrates evidence over a 1.4 second window: a delayed frame postpones commitment by approximately one cycle rather than altering the predicted letter.

5.3.4 Interpretation

That the system remains usable on a saturated budget device is the more consequential result, supporting the accessibility premise of this work while identifying the clearest target for optimisation: at a 94.7% duty cycle the A22 has no headroom, and sustained use would incur significant battery and thermal cost.

On footprint, the contributed classifier's 28,136 bytes should be read against the approximately 7.5 MB MediaPipe landmark model it depends on, fetched once at load and cached. The classifier is a negligible addition to an existing dependency rather than the whole of the system's storage cost.

5.4 Discussion

5.4.1 On comparison with published work

The 75.8-point gain reported here sits beside the 0.8 to 1.4 points that Shi et al. [2] obtain from language model integration (their Table 3, on the development set: 41.4% to 42.8% letter accuracy for the connectionist temporal classification model and 35.9% to 36.7% for the encoder–decoder, in both cases over a baseline that already applies beam search and an insertion penalty). The difference requires explanation.

The two results measure different things under different conditions. Shi et al. operate on an open vocabulary, evaluate by letter error rate over continuous naturally produced fingerspelling, and begin from a recogniser that already exploits temporal context. The difference in difficulty is visible in their baseline: 41.4% letter accuracy on unsegmented video from the wild, against 71.1% here on segmented still images of a closed alphabet. The present work operates on a closed 7,228-word dictionary, evaluates exact-word accuracy, and begins from a per-frame

classifier with no temporal model. Every one of those differences increases the leverage available to lexical constraint. The large gain reported here reflects an easier problem, not a superior method, and it would be a serious misrepresentation to present it otherwise.

What the result does establish is narrower and still worth stating: when fingerspelled input is known to fall within a bounded vocabulary — names from a contact list, terms from a domain glossary, words from a lesson — a correction layer calibrated on the classifier's own confusion statistics converts an unusable letter stream into reliable words without any improvement to the classifier.

5.4.2 On whether the error pattern generalises

It would strengthen the argument if the confusions measured here were the same ones reported elsewhere, since that would make them a property of the alphabet rather than of this pipeline. They are not. Shi et al. [2] list their most frequent substitutions as u → r, o → e, y → i, w → u and j → i, attributing three of them to letters that are rare in their training data. The dominant confusions measured here, in Table 5.2, are M → T, G → Z, C → D, N → T, O → D and X → D. The two sets do not overlap at all.

The two systems differ in nearly every respect that could matter — video against still images, an open vocabulary against a closed one, a sequence model against a per-frame classifier, and different training distributions — so the divergence is not surprising. What it establishes is a constraint on the method rather than support for it: the confusion set is a property of a particular classifier trained on particular data, not of the ASL alphabet under monocular observation.

That constraint is the reason the correction layer is calibrated on the measured confusion matrix of the classifier it sits behind, rather than on any published or assumed set of visually similar letters. It also means the calibration does not transfer: a different classifier, or the same classifier retrained on different data, would require the confusion statistics to be measured again. The procedure generalises; the numbers in Table 5.2 do not.

Chapter 6

Conclusion

6.1 Summary

This project set out to build a fingerspelling recognition system that works on an ordinary phone. It found that the standard approach — train a classifier, report accuracy on a held-out split — produces figures that do not survive contact with data collected elsewhere, and that this failure resists the obvious remedies.

The response was to stop treating the classifier's errors as a defect to be eliminated and start treating them as a channel to be modelled. A correction layer weighted by measured confusion statistics and by classifier confidence raised in-vocabulary word accuracy from 15.9% to 91.7% without any improvement to the underlying classifier, which remains at 71.1% on letters. The deployed system runs at approximately ten inference cycles per second on both a flagship and a budget phone.

Against the objectives of Section 1.3: objectives 1, 2, 4 and 5 were met. Objective 3 was met in that the techniques were evaluated, though the answer was negative — none closed the gap.

6.2 Limitations

The word-level metric is in-vocabulary. Target words are drawn from the system's own dictionary. Words outside it cannot be produced. The reported

figures are an upper bound on open-vocabulary performance and should not be read as a general word accuracy.

J and Z are dynamic signs represented statically. All three datasets present them as still images, which is a misrepresentation of the signs themselves. Z is among the least reliable letters, and this is a limitation of the data rather than of the classifier.

No user study was conducted. The original project plan included evaluation with participants. This was dropped as the project's focus moved to the generalisation problem, and it is stated here explicitly rather than left to vanish. No claim about usability, learnability or user satisfaction is made anywhere in this dissertation; the on-device measurements substantiate responsiveness only.

The system is a progressive web application, not a natively implemented one. It installs to the home screen, runs offline and is published on Google Play, but recognition runs through the browser pipeline rather than in native code. Whether a native implementation would reduce the landmark extraction times reported in Section 5.3 was not tested.

Rotation is not normalised. The feature representation removes position and scale but not orientation, so a tilted hand yields a different feature vector for the same handshape.

The deployed model does not have the highest letter accuracy measured. Mirror augmentation reaches 77.42% against the deployed configuration's 71.10% ($p = 0.004$). It was not deployed because it confers no measurable advantage in word accuracy and because it doubles the seed-to-seed standard deviation, from 2.49 to 5.19 percentage points, which is a fourfold increase in variance; the reasoning is set out in Section 5.2.3. A reader who considers letter accuracy the primary metric would reasonably prefer the augmented model.

The confidence-quality explanation is untested. The account offered in Section 5.2 for why the network outperforms the forest after correction is consistent with the evidence but was not verified by calibration analysis.

6.3 Further work

The most immediate extension is a calibration analysis of the two classifiers' confidence estimates, which would test the explanation offered for the difference in correction performance directly rather than by inference.

A second follows from Section 5.2.3. Mirror augmentation produces two distinct training regimes depending on initialisation, differing by roughly eighteen points in uncorrected word accuracy. Identifying what separates them — and constraining training to the better one — would yield a classifier both more accurate and no less consistent than the one deployed. The bimodality is the finding; exploiting it by selecting favourable seeds would not be legitimate.

A third, given that end-to-end accuracy proved insensitive to a six-point change in letter accuracy, is to direct attention at the decoder instead. The ceiling near 92% is imposed by the dictionary, the cost function and the exact-match requirement, and that is where remaining gains lie.

A fourth is to relax the closed-vocabulary constraint by combining the lexical model with a character-level language model, allowing plausible out-of-vocabulary strings to survive uncorrected.

A fifth follows directly from the device measurements: at a 94.7% duty cycle the budget handset has no headroom. Reducing capture resolution or substituting a lighter landmark model would address this, and neither would require retraining the classifier.

Finally, the generalisation finding suggests a data contribution rather than a modelling one. If the datasets disagree about what some letters look like, the useful next step is a carefully specified capture protocol with documented handshape definitions, against which existing datasets could be audited.

References

[1] C. A. Padden and D. C. Gunsauls, "How the alphabet came to be used in a sign language," *Sign Language Studies*, vol. 4, no. 1, pp. 10–33, 2003. doi: 10.1353/sls.2003.0026.

[2] B. Shi, A. Martinez Del Rio, J. Keane, J. Michaux, D. Brentari, G. Shakhnarovich, and K. Livescu, "American sign language fingerspelling recognition in the wild," in *2018 IEEE Spoken Language Technology Workshop (SLT)*, pp. 145–152, 2018. doi: 10.1109/SLT.2018.8639639.

[3] N. Pugeault and R. Bowden, "Spelling it out: Real-time ASL fingerspelling recognition," in *IEEE International Conference on Computer Vision Workshops (ICCV Workshops)*, pp. 1114–1119, 2011. doi: 10.1109/ICCVW.2011.6130290.

[4] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1521–1528, 2011. doi: 10.1109/CVPR.2011.5995347.

[5] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device real-time hand tracking," *arXiv preprint arXiv:2006.10214*, 2020. `https://arxiv.org/abs/2006.10214`.

[6] B. Alsharif, E. Alalwany, A. Ibrahim, I. Mahgoub, and M. Ilyas, "Real-time american sign language interpretation using deep learning and keypoint tracking," *Sensors*, vol. 25, no. 7, p. 2138, 2025. doi: 10.3390/s25072138.

[7] B. Shi, A. Martinez Del Rio, J. Keane, D. Brentari, G. Shakhnarovich, and K. Livescu, "Fingerspelling recognition in the wild with iterative visual attention," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. doi: 10.1109/ICCV.2019.00550.

[8] M. D. Kernighan, K. W. Church, and W. A. Gale, "A spelling correction program based on a noisy channel model," in *COLING 1990 Volume 2: Papers Presented to the 13th International Conference on Computational Linguistics*, 1990. `https://aclanthology.org/C90-2036/`.

[9] E. Brill and R. C. Moore, "An improved error model for noisy channel spelling correction," in *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 286–293, 2000. doi: 10.3115/1075218.1075255.

[10] Z. Liu and K. He, "A decade's battle on dataset bias: Are we there yet?," in *International Conference on Learning Representations (ICLR)*, 2025. `https://arxiv.org/abs/2403.08632`.

[11] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. doi: 10.1109/ICCV.2019.00140.

[12] Google, "LiteRT (formerly TensorFlow Lite): On-device machine learning runtime." Developer documentation, 2024. `https://ai.google.dev/edge/litert`.

[13] Akash, "ASL alphabet." Kaggle, 2018. `https://www.kaggle.com/datasets/grassknoted/asl-alphabet`.

[14] M. F. I. Pranto, M. R. Islam, M. A. Akbor, N. Ghosh, M. R. Alam, S. Chaki, and M. M. Islam, "ASL-HG: American sign language hand gesture image dataset." Mendeley Data, V1, 2025. doi: 10.17632/j4y5w2c8w9.1.

[15] B. Garg, M. Kasar, A. Kashyap, A. Vats, G. Sharma, and A. Hange, "SignAlphaSet." Mendeley Data, V3, 2025. doi: 10.17632/8fmvr9m98w.3.

[16] Google, "MediaPipe hand landmarker." Developer documentation, 2024. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker.

Appendix A

Source Code

The complete source code for this project — landmark extraction, classifier training, the generalisation experiments, the correction layer, the evaluation scripts and the deployed application — is available at:

`https://github.com/Faruk-hankanoglu/asl-sign-language-project`

The results reported in this dissertation correspond to tag `v2.1-dissertation`. Alongside the code, the repository holds the three extracted landmark files from which every figure in Chapter 5 can be recomputed, a `requirements.txt` pinning the dependencies, this dissertation in PDF form, and a screencast of the application in operation.

Appendix B

Deployed Application

The application is deployed and publicly accessible at:

`https://asl-master-project.vercel.app`

It may be opened in any modern mobile browser and installed to the home screen from the browser menu. Camera access is required; no data leaves the device.

The application is additionally published on Google Play as *Fingerspell — ASL Reader*:

`https://play.google.com/store/apps/details?id=com.ASLReader`

Both routes run the same recognition pipeline and perform all inference on the device.

Appendix C

Raw On-Device Measurements

The measurement blocks produced by the application's *Copy measurements* control, verbatim. Each was captured after the application reported a stable reading.

iPhone 12 Pro

```
ASL fingerspelling app -- on-device measurements
Captured:  2026-09-26T19:04:20.484Z

Device (user agent):  Mozilla/5.0 (iPhone; CPU iPhone OS 18_7 like
Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.2
Mobile/15E148 Safari/604.1
Screen:  390x844 @ DPR 3
Camera resolution:  480×640
Logical CPU cores:  4

Status:  Reading is stable -- safe to record.

Pipeline throughput (throttled, 80 ms nominal interval):  10.0 fps
Unthrottled capacity (1000 / mean total):  30.1 fps
Landmark extraction, mean:  33.0 ms
Classification, mean:  0.14 ms
Total per frame, mean:  33.2 ms
Total per frame, median:  33.0 ms
Total per frame, 95th percentile:  37.0 ms
Total per frame, worst:  38.0 ms
Inference cycles measured:  304 (34 s elapsed); first 20 discarded
as warm-up; all statistics over a rolling window of the last 120
cycles

Classifier:  7,034 parameters, 28,136 bytes float32 (27.5 KB)
Hand landmark model:  MediaPipe hand_landmarker float16 (~7.5 MB,
fetched at load)
```

Samsung Galaxy A22

```
ASL fingerspelling app -- on-device measurements
Captured:  2026-09-26T19:06:41.779Z

Device (user agent):  Mozilla/5.0 (Linux; Android 10; K)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Mobile
Safari/537.36
Screen:  384x857 @ DPR 2.8125
Camera resolution:  480×640
Logical CPU cores:  8

Status:  Reading is stable -- safe to record.

Pipeline throughput (throttled, 80 ms nominal interval):  10.7 fps
Unthrottled capacity (1000 / mean total):  11.3 fps
Landmark extraction, mean:  82.1 ms
Classification, mean:  1.30 ms
Total per frame, mean:  88.5 ms
Total per frame, median:  82.2 ms
Total per frame, 95th percentile:  110.5 ms
Total per frame, worst:  248.8 ms
Inference cycles measured:  267 (35 s elapsed); first 20 discarded
as warm-up; all statistics over a rolling window of the last 120
cycles

Classifier:  7,034 parameters, 28,136 bytes float32 (27.5 KB)
Hand landmark model:  MediaPipe hand_landmarker float16 (~7.5 MB,
fetched at load)
```

Note on the user-agent strings

The user-agent strings above do not identify the devices correctly, and the discrepancy is a property of the browsers rather than an error in reporting. Chrome reduces the Android user-agent string for privacy and reports `Android 10; K` on all devices irrespective of their actual model and operating system version; the iOS platform token `18_7` is likewise a compatibility value retained alongside the Safari version. The device models and operating system versions stated in Table 5.5 were read from the device settings directly.

Note on the derived landmark figures

The landmark extraction times above (33.0 ms and 82.1 ms) are means over every inference cycle, including cycles in which no hand was present, whereas classi-

fication and end-to-end latency are measured only on cycles where a hand was detected. Because these are different sample populations the three figures do not sum exactly. Table 5.5 therefore reports landmark extraction as end-to-end latency minus classification latency (33.1 ms and 87.2 ms), which places all three quantities on the same set of frames.